PROJET .TP.C _____ *bentaleb wail el mahdi :*

PART 01 du projet :

PARTIE 01 du Projet : Structure Definitions and Basic Functions :

1. Define the Stack structure Usages_Restants and the Queue structure Achats_En_Attente :

- **Stack** : Cette structure est utilisée pour gérer les **Usages_Restants**. , ce qui signifie que le dernier service ajouté à l'abonnement du client est le premier disponible pour être consommé.
- **Queue** : Cette structure est utilisée pour gérer les **Achats_En_Attente**., garantissant que les services achetés par le client sont enregistrés et traités dans leur ordre chronologique exact d'arrivée.

```
clientRecord CreateClientRecord() {
    clientRecord c;
    c.numero_compte = 0;
    strcpy(c.nom, "");
    c.statut_abonnement = false;    "statut": Unknown word.
    c.usages_Restants = InitStack();    "Restants": Unknown word.
    c.achat_en_attente = InitQueue();    "achat": Unknown word.
    c.next = NULL;
    return c;
}
```

- **On déclare les deux variable usage_Restant et achat_en_attente a les fonction primitive initstack et initQueue.**

  les fonction primitive :

- ```
  stack InitStack() {
      stack s; s.top = NULL; return s;
  }
  ```

- ```
  Queue InitQueue() {
      Queue q; q.head = NULL; q.tail = NULL; return q;
  }
  ```

2. Define the ClientRecord structure

```
clientRecord CreateClientRecord() {
    clientRecord c;
    c.numero_compte = 0;
    strcpy(c.nom, "");
    c.statut_abonnement = false;    "statut": Unknown word.
    c.usages_Restants = InitStack();    "Restants": Unknown word.
    c.achat_en_attente = InitQueue();    "achat": Unknown word.
    c.next = NULL;
    return c;
}
```

- **Le numéro de compte est mis à zéro et le nom est initialisé comme une chaîne vide via `strcpy` pour éviter toute donnée résiduelle.**
- **Le pointeur `next` est mis à `NULL`, ce qui permet d'intégrer proprement ce client dans la liste chaînée globale par la suite.**

3.Define the Linked List Node structure:

```
// linked list node
Tabnine | Edit | Test | Explain | Document
clientRNode* CreateNode(clientRecord c) {
    clientRNode* NewNode = (clientRNode*)malloc(sizeof(clientRNode));
    NewNode->client = c;
    NewNode->next = NULL;
    return NewNode;
}
```

- **Nous utilisons la fonction `malloc(sizeof(clientRNode))` pour réserver dynamiquement un espace mémoire suffisant pour stocker un client, pour ne pas limiter la taille**
- **Le nœud reçoit une structure `clientRecord` complète (contenant le nom, le numéro de compte, la pile et la file du client)**
- **Le pointeur `next` est initialisé à `NULL` qui permet de définir la fin de la liste.**

4.Implement the basic functions for stacks and queues :

l'**implémentation** est codée dans `fonction.c` :

- **Pour la Pile (`stack`)** :

`InitStack()` : Pour mettre le sommet à `NULL`.

`StackEmpty()` : Pour vérifier si le client n'a plus de services.

`Push()` : Pour ajouter un nouveau service (empiler).

`Pop()` : Pour consommer un service (dépiler).

- **Pour la File (`Queue`)** :

`InitQueue()` : Pour initialiser les pointeurs de tête et de queue.

`EnQueue()` : Pour ajouter un service consommé à l'historique (enfiler).

`UnQueue()` : Pour retirer un élément de la file (défiler).

5.Implement the functions:

- fonction `createcleintrecord` , Alloue l'espace pour un client et initialise ses champs (nom vide, compte à 0, pile et file vides).

```
clientRecord CreateClientRecord() {
    clientRecord c;
    c.numero_compte = 0;
    strcpy(c.nom, "");
    c.statut_abonnement = false;    "statut": Unknown word.
    c.usages_Restants = InitStack();    "Restants": Unknown word.
    c.achat_en_attente = InitQueue();    "achat": Unknown word.
    c.next = NULL;
    return c;
}
```

- **fonction** CreateNode ,Crée un maillon pour la liste chaînée en utilisant `malloc` pour stocker le record du client

```
clientRNode* CreateNode(clientRecord c) {
    clientRNode* NewNode = (clientRNode*)malloc(sizeof(clientRNode));
    NewNode->client = c;
    NewNode->next = NULL;
    return NewNode;
}
```

- **fonction** Addclient ,Insère un nouveau nœud au début de la liste globale des abonnés,
- Le nœud pointe vers le premier client .

```
Tabnine | Edit | Test | Explain | Document
void AddClient(clientRNode** list, clientRecord c) {
    clientRNode* NEW = CreateNode(c);
    NEW->next = *list;
    *list = NEW;
}
```

- **fonction Modifyclientinfo ,** Parcourt la liste pour trouver le client par son numéro et met à jour son nom ou son statut.
- **La fonction** commence par parcourir la liste chaînée à l'aide d'une boucle `while`. Elle utilise un pointeur temporaire (`temp`) pour examiner chaque nœud un par un jusqu'à trouver le numéro de compte (`numero_cmp`) qui correspond à celui que tu as saisi.
- **Pour le Changement de nom** on utilise `strcpy` pour remplacer l'ancien nom par le nouveau (`newName`).

```
Tabnine | Edit | Test | Explain | Document
void ModifyClientInfo(clientRNode** list, int numero_cmp, char* newName, int newstatus) {
    clientRNode* temp = *list;
    while (temp != NULL) {
        if (temp->client.numero_compte == numero_cmp) {
            if (newName != NULL) strcpy(temp->client.nom, newName);
            if (newstatus >= 0) temp->client.statut_abonnement = newstatus;    "statut": Un
            printf(" UpDated !\n");
            return;
        }
        temp = temp->next;
    }
    printf(" not UpDated !\n");
}
```

- **fonction SearchClient ,**Recherche un client dans la liste en utilisant soit son nom, soit son numéro de compte**.**
- La fonction effectue une recherche d'un client de deux manières différentes :
- **Par le Nom** : Elle utilise `strcmp` pour comparer le nom saisi avec celui stocké dans le record du client.
- **Par le Numéro de Compte** : Elle vérifie si l'identifiant numérique correspond

```
clientRNode* SearchClient(clientRNode* list, char* name, int numerocompte) {    "numerocompte": Unkn
    clientRNode* temp = list;
    while (temp != NULL) {
        if ((strcmp(temp->client.nom, name) == 0) || (temp->client.numero_compte == numerocompte)) {
            printf("found!\n");
            return temp;
        }
        temp = temp->next;
    }
    printf("not found !\n");
    return NULL;
}
```

- **fonction CheckSubscription,**Vérifie dans la liste si un client spécifique possède un abonnement actif (renvoie `true` ou `false`).
- une boucle `while` pour parcourir la liste chaînée des clients.
- la fonction compare le numéro de compte fourni en paramètre (`numero_cmp`) avec celui de chaque client dans la liste.
- Une fois le client trouvé , Elle se contente de lire la valeur de la variable `statut_abonnement`.
- Si `statut_abonnement` vaut `1` , la fonction s'arrête et renvoie **true**.
- Si le client n'est pas trouvé ou si son statut est à `0`, elle renvoie **false**

```
bool CheckSubscription(clientRNode* list, int numero_cmp) {
    clientRNode* temp = list;
    while (temp != NULL) {
        if (temp->client.numero_compte == numero_cmp) {
            return temp->client.statut_abonnement;    "statut": U
        }
        temp = temp->next;
    }
    return false;
}
```

6. All structures and functions must be implemented in a separate library :

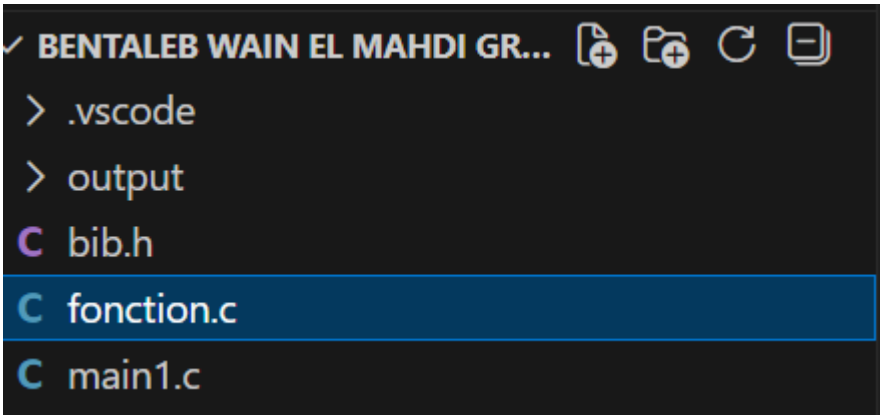
- *Pour ce projet, j'ai découpé le code en trois parties distinctes afin d'assurer une meilleure lisibilité et une maintenance facilitée:*

- **Le fichier `bib.h`** : Ce fichier est le pivot du projet. Il contient les définitions de toutes les structures de données (Piles, Files, Clients) et les prototypes des fonctions. C'est dans ce fichier que nous **appelons** les fonctionnalités logiques pour qu'elles soient accessibles partout dans le programme.

- **Le fichier `fonction.c` (Implémentation)** : Ce fichier contient le développement complet de toutes les fonctions logiques. C'est ici que sont écrites les primitives des structures de données ainsi que les fonctions de gestion des abonnés (ajout, recherche, modification).

- **Le fichier `main1.c` (Programme principal)** : Il contient la boucle principale et l'interface utilisateur. Il utilise les fonctions définies dans `bib.h` sans avoir à connaître les détails de leur implémentation.

- La séparation du code ma permet également d'**optimiser le temps d'exécution et de compilation**. Grâce à la **compilation séparée**, si l'on modifie uniquement l'interface dans `main1.c`, le compilateur n'a pas besoin de re-traiter toute la logique complexe de `fonction.c`. Cela rend le cycle de développement beaucoup plus rapide et efficace.
- **Pour l 'exécution :** J'ai réalisé ce projet en utilisant l'éditeur **VS Code** Pour permettre la compilation de plusieurs fichiers `.c` simultanément, j'ai configuré l'environnement en modifiant les fichiers de paramètres **JSON** pour inclure le chemin du compilateur GCC et rajouter cette commande :
  gcc *.c -o bentaleb_wail_program



PARTIE 02 du Projet :Managing Limited Access and Services (Stack – Queue)

1. Client Création :

la partie du code :

```
clientRNode* CreateNode(clientRecord c) {
    clientRNode* NewNode = (clientRNode*)malloc(sizeof(clientRNode));
    NewNode->client = c;
    NewNode->next = NULL;
    return NewNode;
}

Tabnine | Edit | Test | Explain | Document
void AddClient(clientRNode** list, clientRecord c) {
    clientRNode* NEW = CreateNode(c);
    NEW->next = *list;
    *list = NEW;
}
```

La partie d ‘exécution :

```
--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 1
Enter number of clients: 2
Name of client 1: bentaleb
Added with ID: 1
Name of client 2: EL-mahdi
Added with ID: 2
```

2. We want to subscribe a client to a premium service offering N limited usages + display list:

- **la partie du code :**

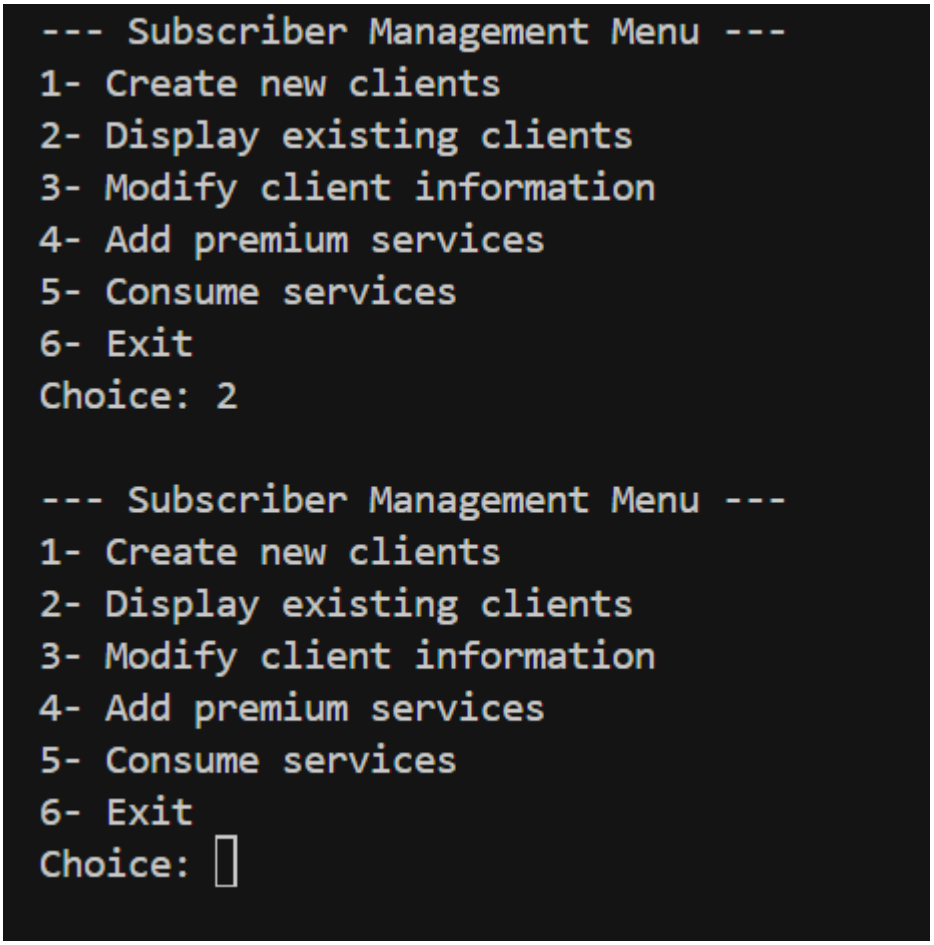
```c
void DisplayList(clientRNode* list) {
    clientRNode* temp = list;
    while (temp != NULL) {
        printf("\n Name: %s", temp->client.nom);
        printf("\n Account: %d", temp->client.numero_compte);
        printf("\n Subscription status: %d\n", temp->client.statut_abonnement);
        temp = temp->next;
    }
}
```

```c
void SubscribeToPremiem(clientRNode* list, char* name, int numaccount, int N, char services[][50]) {
    clientRNode* found = SearchClient(list, name, numaccount);
    if (found != NULL) {
        found->client.statut_abonnement = true;    "statut": Unknown word.
        for (int i = 0; i < N; i++) {
            Stack(&found->client.usages_Restants, services[i]);    "Restants": Unknown word.
        }
    } else {
        printf("Client not found !\n");
    }
}
```

- **la partie exécution : si il n"ya aucun client et l'utilisateur choisit un de display le client le code lui demande de rechoisire : (th 2nd pic)**
- **le choix quand le client et existant :**

```
--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 2

 Name: WAIL
 Account: 1
 Subscription status: 0
```



l'exécution de la partie de la souscription :

```
--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 4
ID: 1
Name: WAIL
How many services? 3
Service 1: cook
Service 2: buy
Service 3: sleep
found!

--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 2

 Name: WAIL
 Account: 1
 Subscription status: 1
```

<u>l'exécution de la partie subscribe + display liste dans tout les cas possible</u>

3. For the case when a client wants to access a service :

- **le code :**

```
void UseService(clientRecord* c) {
    char service[50];
    if (StackEmpty(c->usages_Restants)) {    "Restants": Unknown word.
        printf("the client is NOT allowed to use this service\n");
    } else {
        Unstack(&c->usages_Restants, service);    "Restants": Unknown word.
        Queueq(&c->achat_en_attente, service);    "Queueq": Unknown word.
        printf("the client is allowed to use the service: %s\n", service);
    }
}
```

- **l'exécution du code : si il demande de consommer un service mais le client n'existe pas il reponds par not found**

```
--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 5
ID: 1
Name: WAIL
not found !
```

4.Interaction Between Stack and Queue:

- **le code : (la photo se trouve dans la prochaine page)**

```c
void DisplayHistory(Queue* q) {
    if (QueueEmpty(*q)) return;
    char service[50];
    UnQueue(q, service);
    printf("%s\n", service);
    DisplayHistory(q);
}

Tabnine | Edit | Test | Explain | Document
void AskAndConsumeAgain(clientRecord* c) {
    int choice = 1;
    while (choice == 1 && !StackEmpty(c->usages_Restants)) {    "Restants": Unknown word.
        UseService(c);
        if (StackEmpty(c->usages_Restants)) {    "Restants": Unknown word.
            printf("No more usages left.\n");
            break;
        }
        printf("Consume another? (1=yes, 0=no): ");
        scanf("%d", &choice);
    }
    printf("\n History: \n");
    DisplayHistory(&c->achat_en_attente);    "achat": Unknown word.
}
```

- **l'exécution du code :**

```
--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 5
ID: 1
Name: wail
found!
the client is allowed to use the service: sleep
Consume another? (1=yes, 0=no): 0

 History:
sleep
```

-la deuxième partie dans le cas ou il répond par 1:
dans ce cas il avait 2 abonnement donc il a consommer le deuxième automatiquement.

```
--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 5
ID: 1
Name: WAIL
found!
the client is allowed to use the service: buy
Consume another? (1=yes, 0=no): 1
the client is allowed to use the service: cook
No more usages left.

 History:
buy
cook
```

-dans cette partie ou le client n'as pas de service a consommer il le re demande de choisir automatiquement :

```
--- Subscriber Management Menu ---
1- Create new clients
2- Display existing clients
3- Modify client information
4- Add premium services
5- Consume services
6- Exit
Choice: 5
ID: 1
Name: wail
found!

 History:
```